

Artificial Intelligenc... · [Follow publication](#)

You're reading for free via [CodeWithYog's](#) Friend Link. [Upgrade](#) to access the best of Medium.

★ Member-only story

10 Most Important AI Concepts You Should Understand Before You Start Building AI

A beginner-friendly guide for developers who want to actually understand what they are building

8 min read · Apr 2, 2026



CodeWithYog

Follow



Listen



Share



More



[Non medium members, click here...](#)

Today, let's begin with the fundamentals of AI engineering. There are numerous terms:

LLM, agents, vector databases, tokens, embeddings, RAG, and fine-tuning. Additionally, the majority of tutorials skip over the basics and start building chatbots right away.

The truth is simple:

AI becomes much easier once you understand the core concepts.

You don't need a PhD.

You just need to understand how the system works step by step.

In this post, we'll go through **10 most important AI concepts** in a simple way, and I'll also include platforms where you can try and learn these concepts yourself.

Let's start.

1. Tokens — How AI Actually Reads Text

An AI model does not read a sentence the same way we do before it can comprehend it. The sentence is first divided into tokens, which are tiny fragments.

Humans read sentences as complete thoughts.

AI reads text piece by piece. Word by word, sometimes even part of a word. So when you write a sentence, the model is not really seeing a sentence.

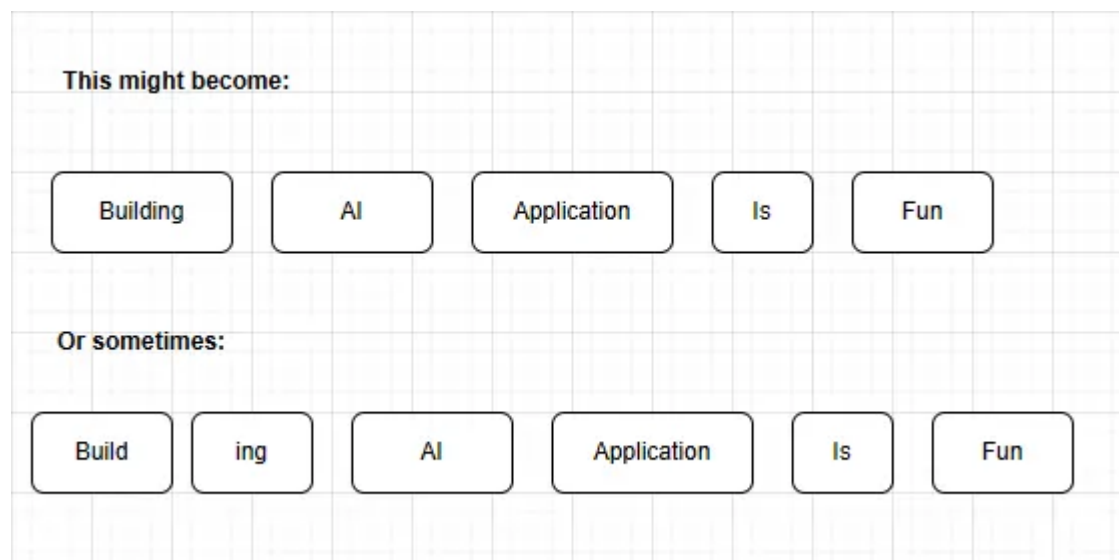
It is seeing a sequence of tokens.

A token can be:

- A full word
- Part of a word
- Punctuation
- Numbers

For example:

“Building AI applications is fun”



Everything in AI is measured in tokens:

- Input size
- Output size
- Pricing

- Context window
- Memory

So when an API says:

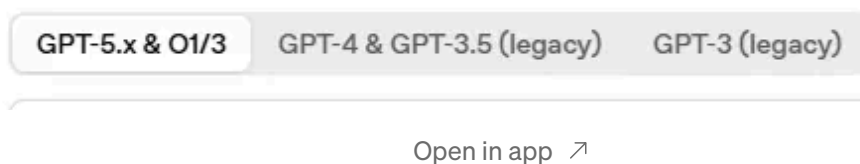
Cost per 1,000 tokens

It means how much text you send and receive.

Try Tokenization Yourself

You can try tokenization here:

- [OpenAI Tokenizer](#)



≡ Medium



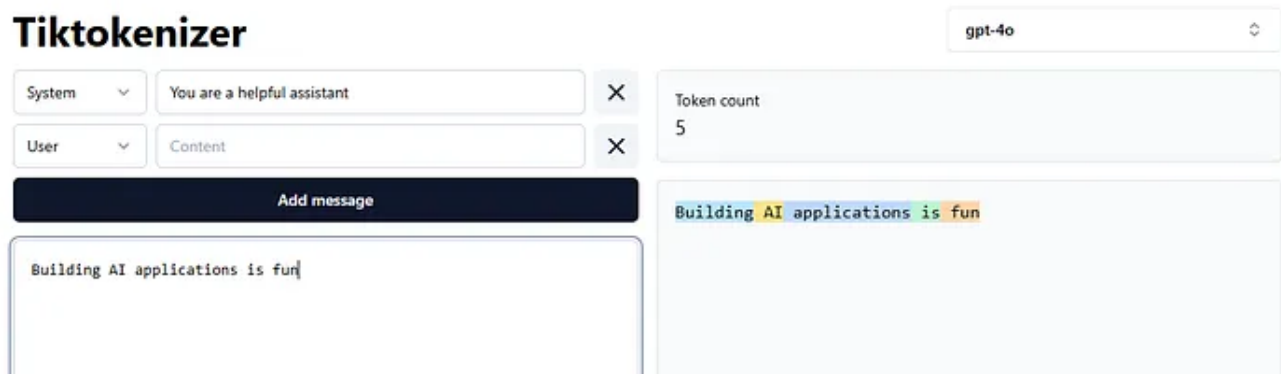
b

Clear Show example

Tokens	Characters
15	61

Building AI applications is fun
be in touch with @CodeWithYog

- [Tiktokenizer](#)



The image shows a web application titled "Tiktokenizer". At the top right, there is a dropdown menu set to "gpt-4o". Below this, there are two input fields: "System" with a dropdown arrow and the text "You are a helpful assistant", and "User" with a dropdown arrow and the text "Content". Each input field has a close button (X) to its right. Below these fields is a dark blue button labeled "Add message". To the right of the input fields, there is a box labeled "Token count" showing the number "5". At the bottom, there is a large text area containing the sentence "Building AI applications is fun". To the right of this text area, the same sentence is displayed with individual words highlighted in different colors (Building: yellow, AI: blue, applications: green, is: orange, fun: red).

Paste any sentence and see how AI breaks it into tokens.

Once you try this, you will start understanding prompts differently.

2. Embeddings — How AI Understands Meaning

After tokenization, text is converted into **numbers**.

These numbers are called **embeddings**.

Embeddings represent meaning mathematically.

Imagine a huge map where:

- Words with similar meaning are close
- Words with different meaning are far apart

For example:

- King → Queen
- Doctor → Nurse
- Apple → Banana
- Apple → Microsoft (company context)

Embeddings are used in:

- Semantic search
- Recommendation systems
- Document search
- Chat with PDF

- RAG systems
- Vector databases

AI does not understand meaning like humans.

It understands **distance between vectors**.

```
import torch
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("all-MiniLM-L6-v2")

# Provide documents in text as input
corpus = [
    "Put your input text",
]

corpus_embeddings = embedder.encode_document(corpus, convert_to_tensor=True)

# Query sentences:
queries = [
    "Put your queries here",
]

# Get 2 sentences of the input document for each query sentence based on cosine
top_k = min(2, len(corpus))
for query in queries:
    query_embedding = embedder.encode_query(query, convert_to_tensor=True)

    # Used cosine-similarity and torch.topk to find the highest 2 scores
    similarity_scores = embedder.similarity(query_embedding, corpus_embeddings)
    scores, indices = torch.topk(similarity_scores, k=top_k)

    print("\nQuery:", query)
    print("Top 2 most similar sentences in corpus:")

    for score, idx in zip(scores, indices):
        print(f"(Score: {score:.4f})", corpus[idx])
```

Now it's quite easy to see that the top three scores represent the most relevant answers when using the “all-MiniLM-L6-v2” embedder.

Try Embeddings Yourself

You can try embeddings here:

- [OpenAI Embeddings Guide](#)
- [HuggingFace Models](#)
- [Sentence Transformers Demo](#)

Search similar sentences and see how embeddings work.

3. Attention — How AI Understands Context

This concept changed AI completely. The meaning of a word depends on context.

Example:

She bought shares in Apple

She ate an apple

Same word.

Different meaning.

The model's ability to examine every word in a sentence and determine what is most important is made possible by attention.

The model looks at the entire sentence at once and begins to connect the words that are related to each other, as opposed to reading one word at a time like humans do. It's similar to reading a sentence and knowing who did what right away rather than taking your time decoding every word.

This is why modern AI can:

- Understand long sentences
- Write code
- Summarize documents
- Translate languages
- Answer questions

Attention is one of the most important ideas in AI applications.

If you want to read the original paper: [“Attention Is All You Need”](#)

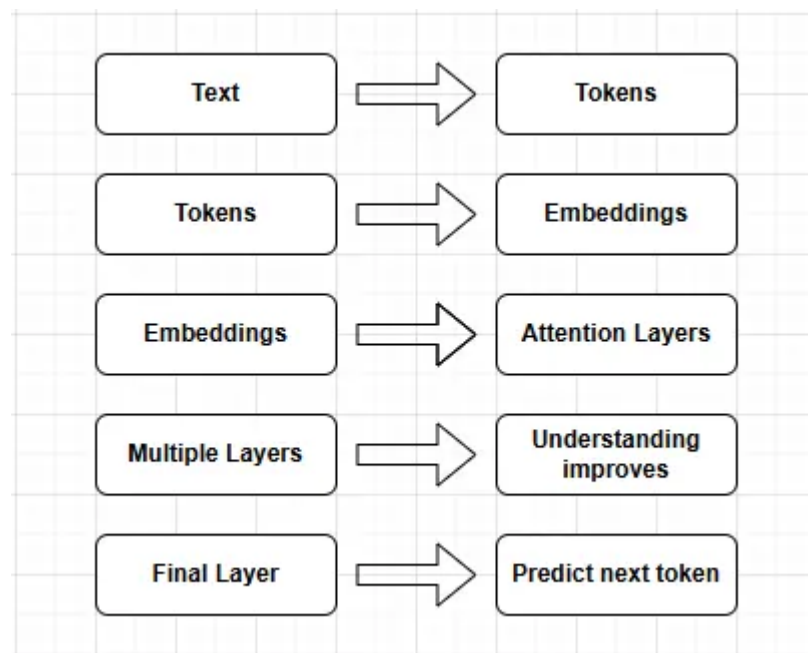
This paper introduced transformers and changed AI forever.

4. Transformers — The Engine Behind Modern AI

Transformers are the architecture used in:

- GPT
- Claude
- Gemini
- Llama
- Mistral
- Most modern AI models

The pipeline looks like this:

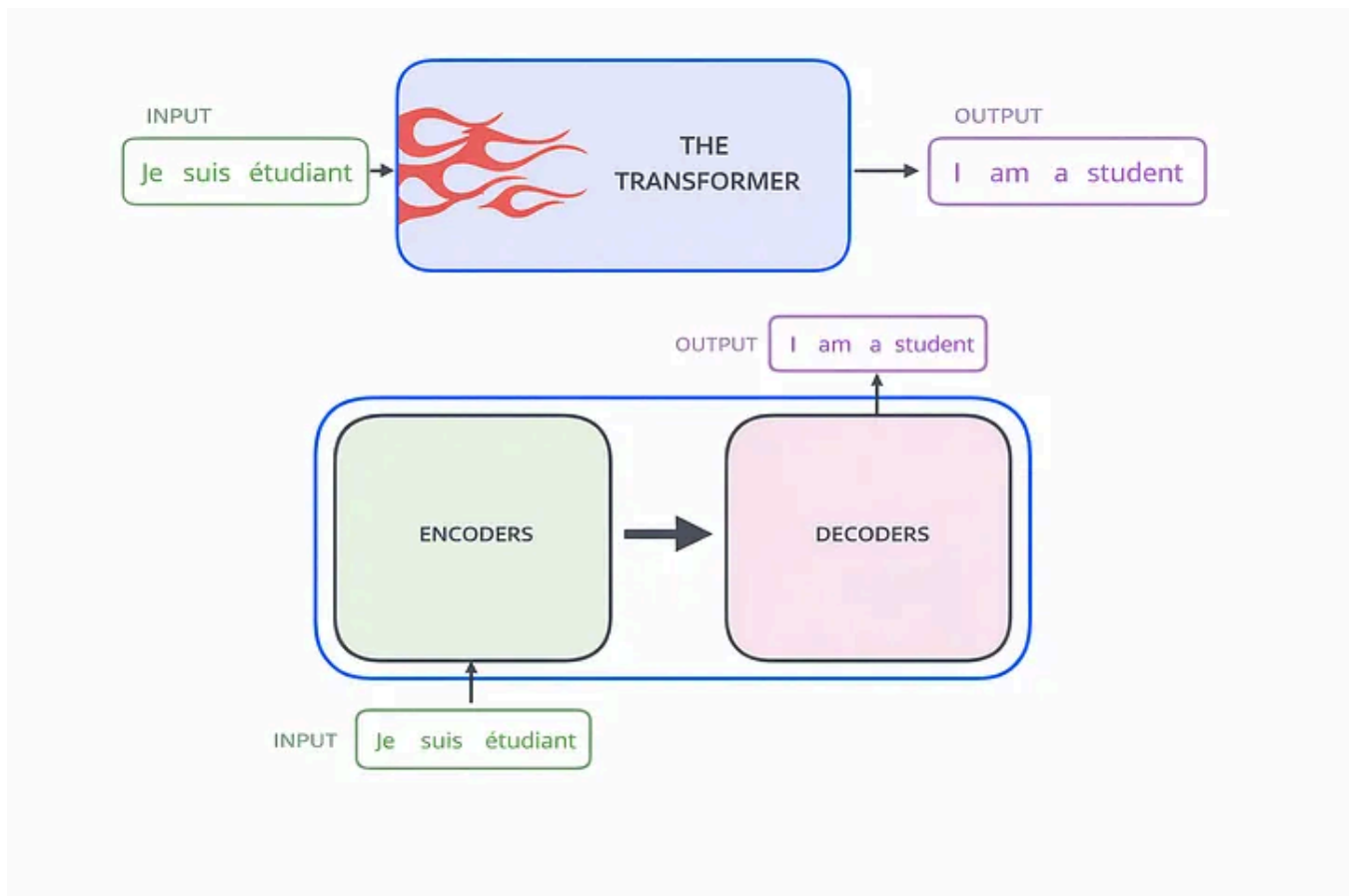


The model does not generate a full sentence at once.

It generates text like this:

- Predict next token
- Add it
- Predict next token again
- Repeat

So when you chat with AI, it is basically predicting the next word again and again very fast.



Learn Transformers Here

- [Illustrated Transformer](#)
- [HuggingFace Course](#)

The illustrated transformer article is one of the best resources on the internet.

5. Large Language Models (LLMs)

Now we connect everything.

A Large Language Model is simply:

A transformer trained on massive amounts of text to predict the next token.

During training, the model sees:

- Books
- Websites

- Code
- Articles
- Documentation
- Conversations

And it learns patterns in language.

It does not store answers like a database.

It learns patterns.

That is why it can:

- Write code
- Explain concepts
- Translate language
- Summarize text
- Generate ideas
- Answer questions

But remember something very important:

LLMs are pattern prediction machines, not truth machines.

Which leads to the next concept.

6. Hallucination — When AI Sounds Right But Is Wrong

AI occasionally provides responses that appear to be well-written and confident, but the information is inaccurate. We refer to this as a hallucination.

This does not imply that the AI is lying. It simply indicates that the model is forecasting what seems accurate rather than verifying what is true.

It might:

- Create fake research papers
- Suggest APIs that don't exist

- Give wrong code
- Make up facts
- Invent statistics

This happens because the model is not trying to tell the truth.

It is trying to generate the most probable next text.

So if something sounds like a correct sentence, it might generate it even if it is wrong.

This is why AI systems in production use:

- RAG
- Verification
- Tool usage
- Human review
- Guardrails

Understanding hallucination is very important before building AI systems.

7. Temperature — Controls Creativity vs Accuracy

AI does not truly “think” of a sentence before writing it. Rather, it gradually predicts the next most likely token.

For every next token, the model calculates probabilities.

Temperature controls how random that selection is.

Low temperature → safer, more accurate, more predictable answers.

High temperature → more creative, more varied, sometimes more risky answers.

So in simple words, **temperature is like a creativity knob for AI.**

Low temperature:

- More accurate
- More predictable

- Good for code and facts

High temperature:

- More creative
- More diverse
- Good for writing and ideas

Typical usage:

- 0.2 → Coding
- 0.4 → Technical answers
- 0.7 → Normal writing
- 1.0 → Creative writing

This one parameter can completely change output style.

8. Context Window — AI Memory

AI models do not remember everything.

They only see what is inside the **context window**.

Context includes:

- Your prompt
- Conversation history
- Documents
- Model responses

If the conversation becomes too long, old information gets removed.

This is why sometimes AI forgets earlier instructions.

Large context windows allow:

- Long conversations

- Large documents
- Codebase analysis
- Chat with PDF
- Knowledge assistants

But larger context also means:

- More memory
- More cost
- Slower responses

So context size is always a trade-off.

9. RAG — Retrieval Augmented Generation

When developing AI applications, this is one of the most crucial ideas.

Rather than relying solely on what it was trained on, the AI first looks for pertinent documents before using those documents to provide a response.

So the process becomes very simple:

First, **retrieve information**.

Then, **generate the answer**.

Because of this, the AI's answers become **more accurate, more up-to-date, and more reliable**, since it is not just relying on memory — it is using real data before responding.

Flow:

1. User asks question
2. Convert question into embedding
3. Search vector database
4. Retrieve relevant documents
5. Send documents to model

6. Model generates answer

This reduces hallucination and allows AI to use your own data.

Most real AI applications use RAG:

- Chat with PDF
- Company knowledge bots
- Documentation assistants
- Support bots
- Internal search tools

Learn RAG Here

- [LangChain Docs](#)
- [LlamaIndex](#)

If you want to build real AI apps, you must learn RAG.

10. AI Agents — AI That Can Take Actions

Normal AI:

You ask → AI answers

AI Agent:

You ask → AI plans → Uses tools → Executes → Checks → Repeats → Finishes task

Agents can:

- Call APIs
- Write code
- Run code
- Search internet
- Read files
- Update database

- Send emails
- Automate workflows

Agents work in a loop:

1. Understand goal
2. Plan steps
3. Execute step
4. Check result
5. Repeat

Agents are powerful but difficult to make reliable.

Learn Agents Here

- [AutoGen](#)
- [CrewAI](#)
- [Semantic Kernel](#)
- [LangGraph](#)

Agents are where AI is heading now.

Final Thoughts

If you understand above 10 concepts, AI will suddenly start making sense. AI stops looking like magic and starts looking like engineering.

And that is when you actually start building real systems.

If You Want to Start Learning AI Properly, Follow This Order

This order works very well:

1. Tokens
2. Embeddings
3. Transformers
4. LLM basics

5. Prompt engineering
6. RAG
7. Vector databases
8. Fine tuning
9. Agents
10. AI system architecture

Follow this path and AI will become much easier to understand... [CodeWithYog](#)

A message from our Founder

Hey, [Sunil](#) here. I wanted to take a moment to thank you for reading until the end and for being a part of this community. Did you know that our team run these publications as a volunteer effort to over 3.5m monthly readers? We don't receive any funding, we do this to support the community.

If you want to show some love, please take a moment to follow me on [LinkedIn](#), [TikTok](#), [Instagram](#). You can also subscribe to our [weekly newsletter](#). And before you go, don't forget to clap and follow the writer!

[Artificial Intelligence](#)[Machine Learning](#)[LLM](#)[AI Agent](#)[Programming](#)[Follow](#)

Published in Artificial Intelligence in Plain English

43K followers · Last published 17 hours ago

New AI, ML and Data Science articles every day. Follow to join our 3.5M+ monthly readers.

[Follow](#)

Written by CodeWithYog

1.8K followers · 69 following

🔧 Full-stack dev (AI/React/.NET/Azure) 🎓 | Educator 📖 | Spirit & code craftsman 🧘‍♂️ IN Building scalable apps, mentoring minds, meditating on middleware.

Responses (13)



Datachamp

What are your thoughts?



Chris AI Studio
Apr 9



RAG is the only way to fix hallucinations in a business setting. Relying on an LLM's "memory" is a gamble, but the RAG pull from your live, proprietary data before generating an answer: you shift from statistical guessing to verified retrieval. Don't train a brain that forgets but build a system that looks it up.



5



1 reply

[Reply](#)



B Dubs
Apr 2



This was awesome - thank you so much!



3



1 reply

[Reply](#)



Fajar Diyos
Apr 13



Great Explanation. Thank you



4



1 reply

[Reply](#)

See all responses
